

NOTES FROM GIT TRAINING

Git & GitHub

My notes, in the order I use the commands

Setup → commands → branches → rebase → pull requests

The daily team workflow is on the first pages, in yellow

Common errors and a one-page cheat sheet are at the end



J Deepthi

October 2026

Contents

Daily development workflow	3
Part 1 • Setup	5
Step 1 • Understand Git and GitHub	5
Step 2 • Install Git	6
Step 3 • First-time configuration	6
Step 4 • Connect your computer to GitHub	7
Step 5 • Create or clone a repository	8
Part 2 • Commands	9
Step 6 • The core loop: status, add, commit	9
Step 7 • Ignore files with .gitignore	10
Step 8 • Inspect history and differences	11
Step 9 • Undo mistakes	12
Step 10 • Remotes: push, fetch, pull	13
Step 11 • Put a project on GitHub	14
Part 3 • Branches	16
Step 12 • Branches	16
Step 13 • Merging and resolving conflicts	17
Step 14 • Stash: park unfinished work	18
Step 15 • Tags and versions	18
Part 4 • Rebase	19
Step 16 • Rebasing: concepts and commands	19
Step 17 • Rebase before a pull request	21
Part 5 • Pull requests and GitHub	23
Step 18 • Collaborate: fork, branch, pull request	23
Step 19 • Issues, reviews and repository settings	24
Step 20 • GitHub CLI (gh)	25
Step 21 • GitHub Actions (CI/CD)	26
Step 22 • Other GitHub features	27
Part 6 • Advanced tools and reference	28
Step 23 • Advanced Git tools	28
Step 24 • Team workflows and conventions	30
Step 25 • Common errors and how to fix them	30
Step 26 • One-page cheat sheet	32

BEFORE YOU START

About these notes

I made these notes while going through a Git training. The slides went through the commands one at a time, so I rewrote everything in the order that is easiest for me to learn and use: setup first, then the basic commands, branches, rebase, and pull requests.

The yellow blocks below are the daily workflow from the training. It is the process we are expected to follow when we work together on one repository. The slide stops at the push, so I added the rebase step, the pull request part and the clean-up after the merge.

My first version of the rebase notes had three mistakes. I wrote `git add.` without the space. I committed after the rebase instead of before it. And I listed `git rebase --continue` as a normal step, when it is only needed after a conflict (otherwise Git just answers "No rebase in progress?"). All three are fixed here, and Step 17 explains them in case someone else makes the same ones.

If you find something wrong, open an issue or a pull request on the repository.

FOLLOW THIS EVERY DAY · THE TEAM PROCESS

Daily development workflow

Update main → branch → code and commit → rebase → push → pull request → clean up

Follow the six blocks in order for every task. Blocks 1, 2 and 4 are the commands from the training slide. Blocks 3, 5 and 6 are the ones I added.



1 · Start from the latest main, on a new branch FROM THE TRAINING SLIDE

```
git switch main          # go to main
git pull                 # download the newest code from GitHub
git switch -c feature/login # create your own branch for this task
```

2 · Write code, check it, commit it FROM THE TRAINING SLIDE

```
# Write code
git status              # which files changed?
git diff                # what exactly changed? read it first
git add .               # stage the changes
git commit -m "Add login functionality" # save a snapshot (repeat as you work)
```

3 · Rebase on the latest main, before you push ADDED

```
git fetch origin        # download what teammates merged meanwhile
git rebase origin/main  # replay your commits on top of the latest main
# Only if it stops on a conflict:
# fix the file -> git add <file> -> git rebase --continue
# "Successfully rebased and updated" means it is finished. Nothing more to run.
```

4 · Push your branch to GitHub FROM THE TRAINING SLIDE + ONE ADDED LINE

```
git push -u origin feature/login # first push of the branch
git push --force-with-lease      # later pushes, if you rebased after pushing
```

5 · Open the pull request on GitHub and get it reviewed ADDED

- Press **Compare & pull request**. Base: `main`. Compare: `feature/login`.
- Write what changed and why. Link the issue with `Closes #12`. Request a reviewer.
- Wait for the automatic checks to turn green.
- Review comments: fix the code, then `git add .`, `git commit -m "..."`, `git push`. The same pull request updates itself.
- When approved, merge it with the method your team uses, and delete the branch on GitHub.

6 · Clean up after the merge ADDED

```
git switch main
git pull # main now contains your merged work
git branch -d feature/login # delete the local branch
git fetch --prune # forget branches that were deleted on GitHub
```

Rules to follow strictly when you collaborate

ALWAYS	NEVER
✓ Start every task from an updated <code>main</code> (<code>git switch main</code>, <code>git pull</code>). ✓ One branch per task, with a clear name: <code>feature/login</code>, <code>bugfix/date-parse</code>. ✓ Run <code>git status</code> and <code>git diff</code> before <code>git add</code>, so only intended files go in. ✓ Commit small and often, with a message that says what the commit does. ✓ Rebase on <code>origin/main</code> before pushing and before asking for review. ✓ Run the tests before you push. ✓ Send every change to <code>main</code> through a pull request with at least one review. ✓ Delete the branch after it is merged, and start the next task from <code>main</code> again.	✗ Commit or push directly to <code>main</code>. ✗ Push without first fetching and rebasing on the latest <code>main</code>. ✗ Use plain <code>git push --force</code>. On your own branch use <code>--force-with-lease</code>. ✗ Rebase <code>main</code> or any branch that other people are working on. ✗ Commit passwords, API keys, <code>.env</code> files or large data files. ✗ Mix several unrelated tasks in one branch or one pull request. ✗ Merge your own pull request while checks are failing or before it is reviewed. ✗ Leave a branch for days without rebasing; conflicts grow with every day.

Note: If something goes wrong on the way: rebase conflicts are in Step 16, the rebase and push details in Step 17, pull requests in Step 18, and error messages with their fixes in Step 25.

PART 1

Setup

Steps 1 to 5: what Git is, install, configure, connect to GitHub, get a repository

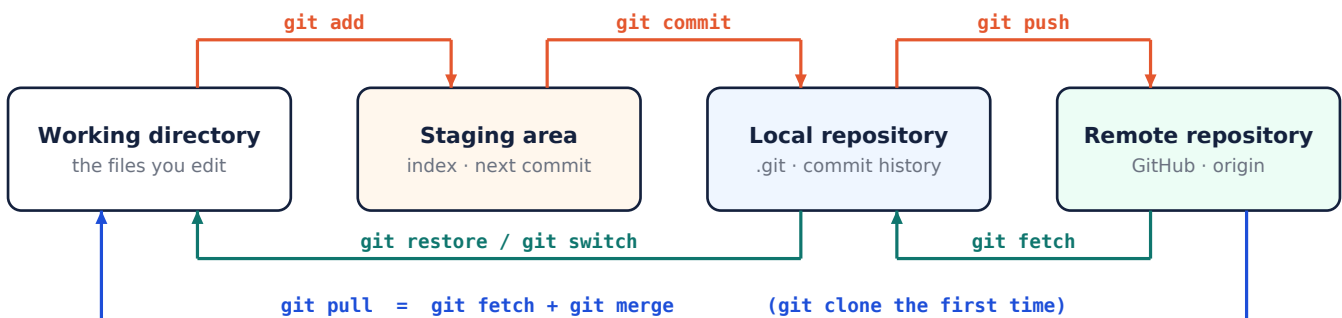
STEP 01

Understand Git and GitHub

Git is a version control system that runs on your own computer. It records snapshots of your project (called commits) so you can see what changed, go back to any earlier version, and work on several ideas in parallel. **GitHub** is a website that stores Git repositories online and adds teamwork features on top: pull requests, issues, code review and automation.

	Git	GitHub
What it is	A command-line tool installed on your machine	A cloud hosting service for Git repositories
Works offline	Yes, everything is local	No, needs internet
Main job	Track history, branch, merge	Share code, review, collaborate, automate
Alternatives	Mercurial, SVN	GitLab, Bitbucket

Where your work lives



A file moves through four states: **untracked** (Git has never seen it), **modified** (changed since the last commit), **staged** (marked to go into the next commit) and **committed** (saved in history).

Vocabulary you will see everywhere

Term	Meaning
Repository (repo)	A project folder tracked by Git. The history lives in the hidden <code>.git</code> folder.
Commit	One saved snapshot of the project, with a message, an author and a unique hash such as <code>a1f3c9e</code> .
Branch	A movable pointer to a line of commits. Lets you work without touching <code>main</code> .
HEAD	Where you are right now: the commit (usually the branch tip) currently checked out.
Staging area (index)	The waiting room for changes that will go into the next commit.
Remote	A copy of the repository on another machine, normally GitHub.
origin	The default name Git gives to the remote you cloned from.

Term	Meaning
upstream	By convention, the original repository that you forked from.
Clone	Download a full copy of a remote repository, including all history.
Fork	Your own copy of someone else's repository on GitHub.
Pull request (PR)	A request on GitHub to merge one branch into another, with review and discussion.
Merge / Rebase	Two ways of combining the work from two branches.

STEP 02

Install Git

<code>winget install --id Git.Git -e</code>	Windows. Or download the installer from git-scm.com . It also installs Git Bash, a terminal where every command in these notes works.
<code>brew install git</code>	macOS with Homebrew. Alternative: <code>xcode-select --install</code> .
<code>sudo apt update && sudo apt install git</code>	Ubuntu / Debian.
<code>sudo dnf install git</code>	Fedora / RHEL.
<code>git --version</code>	Confirm the install and show the version number.

Note: `git switch` and `git restore` need Git 2.23 or newer. If your version is older, update Git, or use the older `git checkout` forms shown beside them.

STEP 03

First-time configuration

Do this once per computer. Git stamps your name and email on every commit you make.

<code>git config --global user.name "Your Name"</code>	Set the author name used in your commits.
<code>git config --global user.email "you@example.com"</code>	Set the author email. Use the same email as your GitHub account so commits link to your profile.
<code>git config --global init.defaultBranch main</code>	Name the first branch <code>main</code> in every new repository (older Git used <code>master</code>).
<code>git config --global core.editor "code --wait"</code>	Use VS Code for commit messages. Other values: <code>nano</code> , <code>vim</code> , <code>notepad</code> .
<code>git config --global core.autocrlf true</code>	Windows: convert line endings automatically. On macOS and Linux use <code>input</code> instead of <code>true</code> .
<code>git config --global pull.rebase false</code>	Make <code>git pull</code> merge (the default). Set <code>true</code> if you prefer rebasing.
<code>git config --global color.ui auto</code>	Coloured output in the terminal.
<code>git config --list</code>	Show every setting currently in effect.
<code>git config --list --show-origin</code>	Same, plus the file each setting comes from.
<code>git config user.email</code>	Print one setting.
<code>git config --global --edit</code>	Open the global config file in your editor.

<code>git config --global --unset core.editor</code>	Remove a setting.
<code>git help commit</code>	Open the full manual for a command. <code>git commit -h</code> gives a short summary.

Level	Flag	Applies to	Stored in
System	<code>--system</code>	Every user on the computer	Git install folder
Global	<code>--global</code>	All your repositories	<code>~/.gitconfig</code>
Local	<code>--local</code> (default)	The current repository only; overrides the others	<code>.git/config</code>

Tip: To keep your email private on GitHub, turn on "Keep my email addresses private" in Settings, Emails, and use the `...@users.noreply.github.com` address GitHub shows you there as `user.email`.

STEP 04

Connect your computer to GitHub

Create a free account at github.com. GitHub does not accept your account password for Git commands (that was removed in August 2021), so choose one of the three methods below.

Method	Repository URL looks like	Good for
GitHub CLI	(handled for you)	The quickest setup. One command and a browser login.
HTTPS + token	<code>https://github.com/USER/REPO.git</code>	Works everywhere, including behind firewalls.
SSH key	<code>git@github.com:USER/REPO.git</code>	No prompts after the one-time setup.

Method 1: GitHub CLI

<code>gh auth login</code>	Sign in through the browser and let <code>gh</code> configure Git credentials for you. Install <code>gh</code> first (Step 20).
<code>gh auth status</code>	Check which account is signed in.

Method 2: HTTPS with a personal access token

- On GitHub: profile picture, **Settings, Developer settings, Personal access tokens**, generate a token. For a classic token tick the `repo` scope; for a fine-grained token choose the repositories and give "Contents: Read and write".
- Copy the token immediately. GitHub shows it only once.
- When Git asks for a password during `git push`, paste the token instead of your password.

<code>git config --global credential.helper manager</code>	Windows: remember credentials with Git Credential Manager (ships with Git for Windows and can sign you in through the browser).
<code>git config --global credential.helper osxkeychain</code>	macOS: store the token in Keychain.
<code>git config --global credential.helper cache</code>	Linux: keep it in memory for 15 minutes. <code>store</code> saves it permanently but in plain text.

Method 3: SSH key

```
ssh-keygen -t ed25519 -C "you@example.com" # create a key pair (press Enter for defaults)
eval "$(ssh-agent -s)" # start the SSH agent
ssh-add ~/.ssh/id_ed25519 # load your private key into the agent
cat ~/.ssh/id_ed25519.pub # print the PUBLIC key, then copy it
# GitHub: Settings > SSH and GPG keys > New SSH key > paste > Add SSH key
ssh -T git@github.com # test: GitHub replies "Hi USER!"
```

Careful: Share only the `.pub` file. The file without `.pub` is your private key: never upload, email or commit it. Treat access tokens the same way.

STEP 05

Create or clone a repository

There are only two ways to get a repository: turn an existing folder into one, or copy one that already exists.

<code>git init</code>	Turn the current folder into a Git repository (creates the hidden <code>.git</code> folder).
<code>git init my-project</code>	Create a new folder called <code>my-project</code> and initialise it.
<code>git clone <url></code>	Download a repository and its full history into a new folder.
<code>git clone <url> my-folder</code>	Clone into a folder name of your choice.
<code>git clone -b dev <url></code>	Clone and check out the <code>dev</code> branch straight away.
<code>git clone --depth 1 <url></code>	Shallow clone: only the latest commit. Fast for big repositories.
<code>ls -a</code>	List files including hidden ones, to confirm <code>.git</code> exists.

Careful: Deleting the `.git` folder deletes the whole history. Never run `git init` inside a folder that is already inside another repository.

STEP 06

The core loop: status, add, commit

This is 80 percent of daily Git. Edit files, check what changed, stage what you want, commit with a message.

```
echo "# My Project" > README.md    # create a file
git status                        # README.md shows as untracked
git add README.md                 # stage it
git commit -m "Add README"         # save the snapshot
git log --oneline                 # see the commit you just made
```

<code>git status</code>	Show the current branch, staged changes, unstaged changes and untracked files.
<code>git status -s</code>	Short form. ?? untracked, A added, M modified, D deleted, R renamed.
<code>git add <file></code>	Stage one file.
<code>git add file1 file2</code>	Stage several files.
<code>git add .</code>	Stage every change in the current folder and below.
<code>git add -A</code>	Stage every change in the whole repository, including deletions.
<code>git add *.py</code>	Stage files matching a pattern.
<code>git add -p</code>	Go through changes piece by piece and choose which parts to stage.
<code>git commit -m "message"</code>	Commit everything that is staged, with a one-line message.
<code>git commit</code>	Commit and open the editor to write a longer message.
<code>git commit -am "message"</code>	Stage all tracked modified files and commit in one step. New files are not included.
<code>git rm <file></code>	Delete a file and stage the deletion.
<code>git rm --cached <file></code>	Stop tracking a file but keep it on disk.
<code>git mv old.py new.py</code>	Rename or move a file and stage the change.

Tip: Write commit messages as short commands: "Add login form", "Fix null check in parser". Keep the first line under about 50 characters.

STEP 07

Ignore files with .gitignore

A `.gitignore` file in the project root lists files Git should never track: secrets, virtual environments, build output, large data, editor settings.

Pattern	Ignores
*.log	Every file ending in <code>.log</code> , in any folder
build/	The whole <code>build</code> folder
/todo.txt	<code>todo.txt</code> in the root only, not in subfolders
docs/*.pdf	PDF files directly inside <code>docs</code>
**/temp	Any file or folder called <code>temp</code> , at any depth
!keep.log	Exception: do not ignore <code>keep.log</code>
# text	A comment line

Example .gitignore for a Python or data project

```
# Python
__pycache__/
*.pyc
.venv/
venv/

# Secrets and local settings
.env
*.key

# Notebooks, data and models
.ipynb_checkpoints/
data/raw/
*.h5
*.pkl

# Editor and OS files
.vscode/
.idea/
.DS_Store
Thumbs.db
```

<code>git status --ignored</code>	Show ignored files as well.
<code>git check-ignore -v <file></code>	Explain which rule causes a file to be ignored.
<code>git rm -r --cached .</code>	Untrack everything (files stay on disk). Follow with <code>git add .</code> and a commit to apply a new <code>.gitignore</code> to files already tracked.
<code>git config --global core.excludesFile ~/.gitignore_global</code>	Use one personal ignore file for all your repositories.

Note: `.gitignore` only affects untracked files. If a file was already committed, remove it from tracking with `git rm --cached <file>` first. Ready-made templates for every language are in the [github/gitignore](#) repository.

Inspect history and differences

History

<code>git log</code>	Full history of the current branch, newest first. Press q to quit.
<code>git log --oneline</code>	One line per commit: short hash and message.
<code>git log --oneline --graph --decorate --all</code>	Draw every branch as a text graph. A good default view.
<code>git log -n 5</code>	Only the last 5 commits.
<code>git log -p</code>	Show the actual changes (patch) of every commit.
<code>git log --stat</code>	Show which files changed and how many lines.
<code>git log --author="Asha"</code>	Commits by one author.
<code>git log --since="2 weeks ago"</code>	Commits after a date. Also --until .
<code>git log --grep="fix"</code>	Commits whose message contains a word.
<code>git log -S"load_data"</code>	Commits that added or removed this exact text in the code.
<code>git log -- <file></code>	History of one file. Add --follow to track it across renames.
<code>git log main..feature</code>	Commits that are on feature but not on main .
<code>git show <commit></code>	Details and full diff of one commit.
<code>git show <commit>:<file></code>	Print a file exactly as it was in that commit.
<code>git blame <file></code>	Show who last changed each line, and in which commit.
<code>git blame -L 10,20 <file></code>	Blame only lines 10 to 20.
<code>git shortlog -sn</code>	Number of commits per author.

Differences

<code>git diff</code>	Changes in the working directory that are not staged yet.
<code>git diff --staged</code>	Changes that are staged, i.e. what the next commit will contain.
<code>git diff HEAD</code>	All uncommitted changes, staged and unstaged together.
<code>git diff <commitA> <commitB></code>	Difference between two commits.
<code>git diff main..feature</code>	Difference between the tips of two branches.
<code>git diff main...feature</code>	Three dots: what feature changed since it split from main .
<code>git diff --stat</code>	Summary only: files and line counts.
<code>git diff --name-only</code>	Just the names of changed files.

Ways to point at a commit

Reference	Means
HEAD	The commit you are on now
HEAD~1, HEAD~3	One commit back, three commits back

Reference	Means
alf3c9e	A specific commit, by the first 7 or more characters of its hash
main, feature/login	The latest commit on that branch
origin/main	Your last known copy of main on the remote
v1.0.0	The commit a tag points to
HEAD@{2}	Where HEAD was two moves ago (from the reflog)

STEP 09

Undo mistakes

Pick the row that matches your situation. Rows are ordered from safest to most destructive.

<code>git restore <file></code>	Throw away unstaged changes in a file. Old form: <code>git checkout -- <file></code> .
<code>git restore .</code>	Throw away all unstaged changes in the current folder.
<code>git restore --staged <file></code>	Unstage a file but keep its changes. Old form: <code>git reset HEAD <file></code> .
<code>git restore --source=HEAD~2 <file></code>	Bring back a file as it was two commits ago.
<code>git commit --amend -m "Better message"</code>	Rewrite the message of the last commit.
<code>git commit --amend --no-edit</code>	Add the currently staged changes to the last commit, same message. Use after <code>git add</code> of a forgotten file.
<code>git revert <commit></code>	Create a new commit that reverses an old one. Safe for commits already pushed.
<code>git revert -m 1 <merge-commit></code>	Reverse a merge commit, keeping the first parent (usually main).
<code>git reset --soft HEAD~1</code>	Undo the last commit, keep its changes staged.
<code>git reset HEAD~1</code>	Undo the last commit, keep its changes unstaged (this is --mixed).
<code>git reset --hard HEAD~1</code>	Undo the last commit and delete its changes.
<code>git reset --hard origin/main</code>	Make your branch identical to the remote (run <code>git fetch</code> first). Local work is lost.
<code>git clean -n</code>	Preview which untracked files would be deleted.
<code>git clean -fd</code>	Delete untracked files and folders. Add x (-fdx) to delete ignored files too.
<code>git reflog</code>	List every position HEAD has been in. Use it to find commits that look lost.
<code>git reset --hard HEAD@{2}</code>	Jump back to a position found in the reflog.

git reset mode	Commit history	Staging area	Working files
--soft	Moved back	Kept	Kept
--mixed (default)	Moved back	Cleared	Kept
--hard	Moved back	Cleared	Overwritten

Careful: `reset` rewrites history, so use it only on commits you have **not** pushed. For commits that are already on GitHub, use `git revert`. `reset --hard` and `clean -f` cannot be undone for uncommitted work.

STEP 10

Remotes: push, fetch, pull

A remote is a named link to a repository on GitHub. `origin/main` is your local record of where `main` was on the remote the last time you talked to it.

Manage remotes

<code>git remote -v</code>	List remotes with their URLs.
<code>git remote add origin <url></code>	Connect the local repository to a GitHub repository under the name <code>origin</code> .
<code>git remote set-url origin <url></code>	Change the URL, for example to switch from HTTPS to SSH.
<code>git remote rename origin github</code>	Rename a remote.
<code>git remote remove origin</code>	Remove the link (nothing is deleted on GitHub).
<code>git remote show origin</code>	Detailed view: branches, tracking and whether you are ahead or behind.

Upload

<code>git push -u origin main</code>	Push <code>main</code> and remember the link, so plain <code>git push</code> and <code>git pull</code> work afterwards. Use it on the first push of any branch.
<code>git push</code>	Push the current branch to the remote branch it tracks.
<code>git push origin <branch></code>	Push a specific branch.
<code>git push --all</code>	Push all local branches.
<code>git push origin --delete <branch></code>	Delete a branch on GitHub.
<code>git push --force-with-lease</code>	Overwrite the remote branch after rewriting history, but only if nobody else pushed in the meantime.

Download

<code>git fetch</code>	Download new commits from the remote but do not change your files or branches.
<code>git fetch --all --prune</code>	Fetch from every remote and forget branches that were deleted on GitHub.
<code>git pull</code>	Fetch and then merge into the current branch.
<code>git pull origin main</code>	Pull a specific branch from a specific remote.
<code>git pull --rebase</code>	Fetch, then replay your local commits on top. Avoids extra merge commits.
<code>git switch <branch></code>	If the branch exists only on the remote, this creates a local copy that tracks it.
<code>git branch -u origin/main</code>	Set which remote branch the current branch tracks.

	git fetch	git pull
Downloads new commits	Yes	Yes
Changes your working files	No	Yes
Use it when	You want to look before you merge	You want to be up to date right now

Careful: Plain `git push --force` can silently delete teammates' commits. Never force-push to `main` or any shared branch; on your own branch prefer `--force-with-lease`.

STEP 11

Put a project on GitHub

Scenario A: you already have a folder on your computer

On GitHub click **New repository**, give it a name, and leave "Add a README" unticked so the repository starts empty. Then:

```
cd my-project
git init                # skip if it is already a repository
git add .
git commit -m "Initial commit"
git branch -M main      # make sure the branch is called main
git remote add origin https://github.com/USERNAME/REPO.git
git push -u origin main # upload and link
```

Scenario B: start on GitHub, then work locally

```
git clone https://github.com/USERNAME/REPO.git
cd REPO
# ... create or edit files ...
git add .
git commit -m "Describe the change"
git push
```

Scenario C: one command with the GitHub CLI

```
gh repo create my-project --public --source=. --remote=origin --push
```

Tip: After the first push, every task follows the **daily development workflow** on the yellow pages at the front of these notes: update `main`, branch, commit, rebase, push, pull request, clean up.

Files every good repository has

File	Purpose
README.md	Front page of the repository: what it is, how to install and run it.
LICENSE	How others may use the code (MIT, Apache-2.0, GPL...). Without one, nobody may legally reuse it.
.gitignore	Files to keep out of Git.
requirements.txt / package.json	Dependencies, so others can reproduce your environment.

File	Purpose
CONTRIBUTING.md	How to contribute: setup, coding style, PR rules.
.github/workflows/*.yml	GitHub Actions automation (Step 21).
.github/PULL_REQUEST_TEMPLATE.md	Text pre-filled into every new pull request.
.github/ISSUE_TEMPLATE/	Forms for bug reports and feature requests.
CODEOWNERS	Who is automatically asked to review which paths, e.g. *.py @username.
SECURITY.md	How to report a vulnerability privately.

PART 3

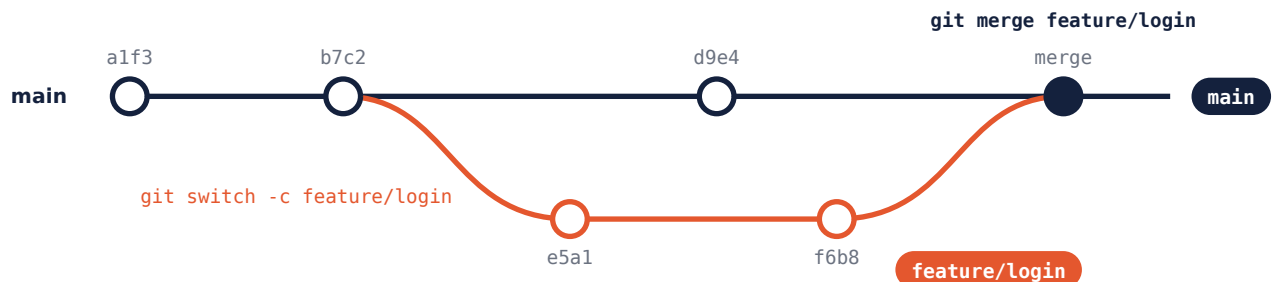
Branches

Steps 12 to 15: branches, merging and conflicts, stash, tags

STEP 12

Branches

A branch lets you develop a feature or fix in isolation. `main` stays stable while you experiment; when the work is ready you merge it back.



<code>git branch</code>	List local branches. The current one has a <code>*</code> .
<code>git branch -a</code>	List local and remote branches. <code>-r</code> shows remote ones only.
<code>git branch -vv</code>	List branches with their last commit and the remote branch each one tracks.
<code>git branch <name></code>	Create a branch (you stay on the current one).
<code>git switch <name></code>	Move to a branch. Old form: <code>git checkout <name></code> .
<code>git switch -c <name></code>	Create a branch and move to it. Old form: <code>git checkout -b <name></code> .
<code>git switch -c <name> <commit></code>	Create a branch starting from a specific commit or tag.
<code>git switch -</code>	Go back to the branch you were on before.
<code>git branch -m <new-name></code>	Rename the current branch.
<code>git branch -m <old> <new></code>	Rename another branch.
<code>git branch -d <name></code>	Delete a branch that is already merged.
<code>git branch -D <name></code>	Force delete, even if it has unmerged work.
<code>git branch --merged</code>	Branches already merged into the current one (safe to delete). Opposite: <code>--no-merged</code> .
<code>git switch --detach <commit></code>	Look at an old commit without being on any branch ("detached HEAD").

Tip: Name branches by purpose: `feature/login`, `bugfix/null-date`, `hotfix/payment`, `docs/readme`. In detached HEAD, run `git switch -c <name>` if you want to keep any commits you make there.

STEP 13

Merging and resolving conflicts

Merging brings the commits of one branch into another. Always switch to the branch that should **receive** the work first.

```
git switch main          # go to the receiving branch
git pull                 # make sure it is up to date
git merge feature/login  # bring the feature in
git branch -d feature/login # delete the branch once merged
```

git merge <branch>	Merge a branch into the current one.
git merge --no-ff <branch>	Always create a merge commit, even when a fast-forward is possible. Keeps the branch visible in history.
git merge --squash <branch>	Combine all the branch commits into one set of staged changes; you then commit once.
git merge --abort	Cancel a merge in progress and return to the state before it.
git merge --continue	Finish the merge after you resolved conflicts.
git log --merges	Show only merge commits.

Merge type	When it happens	Result
Fast-forward	main has no new commits since the branch was created	The main pointer just moves forward. No merge commit.
Three-way merge	Both branches have new commits	Git creates a merge commit with two parents.

Resolving a conflict

A conflict happens when both branches changed the same lines. Git stops and marks the file like this:

```
<<<<<<< HEAD
timeout = 30          (your version, on the current branch)
=====
timeout = 60          (their version, from the branch being merged)
>>>>>>> feature/login
```

- Run **git status** to see the files listed under "Unmerged paths".
- Open each file, keep the correct lines, and delete the three marker lines (<<<<<<<, =====, >>>>>>>).
- Run **git add <file>** for each resolved file.
- Run **git commit** (or **git merge --continue**) to complete the merge.

git diff --name-only --diff-filter=U	List only the files that still have conflicts.
git checkout --ours <file>	Take your version of the whole file.
git checkout --theirs <file>	Take the incoming version of the whole file.
git mergetool	Open a visual merge tool, if one is configured.

STEP 14

Stash: park unfinished work

Stash saves your uncommitted changes on a stack and gives you a clean working directory, so you can switch branches or pull, then bring the changes back.

<code>git stash</code>	Save tracked, uncommitted changes and clean the working directory.
<code>git stash push -m "half-done form"</code>	Stash with a description.
<code>git stash -u</code>	Also stash untracked (new) files.
<code>git stash list</code>	Show all stashes: <code>stash@{0}</code> is the newest.
<code>git stash show -p stash@{0}</code>	Show the changes stored in a stash.
<code>git stash pop</code>	Re-apply the newest stash and remove it from the stack.
<code>git stash apply stash@{1}</code>	Re-apply a stash but keep it on the stack.
<code>git stash drop stash@{0}</code>	Delete one stash.
<code>git stash clear</code>	Delete all stashes.
<code>git stash branch <name></code>	Create a new branch from the stash and apply it there.

STEP 15

Tags and versions

A tag is a permanent name for one commit, normally used to mark releases. Version numbers usually follow semantic versioning: `MAJOR.MINOR.PATCH`, for example `v2.4.1`.

<code>git tag</code>	List all tags.
<code>git tag v1.0.0</code>	Lightweight tag on the current commit.
<code>git tag -a v1.0.0 -m "First release"</code>	Annotated tag with message, author and date. Recommended for releases.
<code>git tag -a v0.9.0 <commit></code>	Tag an older commit.
<code>git show v1.0.0</code>	Show the tag details and the commit it points to.
<code>git push origin v1.0.0</code>	Push one tag to GitHub. Tags are not pushed by a normal <code>git push</code> .
<code>git push origin --tags</code>	Push all tags.
<code>git tag -d v1.0.0</code>	Delete a tag locally.
<code>git push origin --delete v1.0.0</code>	Delete a tag on GitHub.
<code>git switch --detach v1.0.0</code>	Check out the code exactly as it was at that tag.
<code>git describe --tags</code>	Describe the current commit relative to the nearest tag.

STEP 16

Rebasing: concepts and commands

Rebase means "give my branch a new base". Git lifts the commits that exist only on your branch, moves the branch to the tip of another branch, and replays your commits there one by one. The result is a straight-line history, as if you had started your work from the latest `main`.



What Git does, step by step

- Finds the commit where the two branches split (B in the picture).
- Sets aside every commit you made after that point (D, E) as a list of patches.
- Moves your branch pointer to the tip of the target branch (C).
- Re-applies the patches in order. Each one becomes a **new commit** (D', E') with the same changes and message but a different hash, because its parent is different.
- The old D and E are no longer on any branch. They stay recoverable through `git reflog` for a few weeks.
- `main` itself has not moved. To finish, switch to `main` and merge the feature: it will be a simple fast-forward.

```
git switch feature/login
git rebase main           # replay my commits on top of the latest main
git switch main
git merge feature/login   # fast-forward: no merge commit is created
```

Everyday rebase commands

<code>git rebase <branch></code>	Replay the current branch on top of another branch.
<code>git rebase --continue</code>	Carry on after you resolved a conflict.
<code>git rebase --skip</code>	Drop the commit that is causing the conflict and move on.
<code>git rebase --abort</code>	Cancel everything and return to the state before the rebase.
<code>git rebase --autostash main</code>	Stash uncommitted changes first, rebase, then restore them.
<code>git pull --rebase</code>	Fetch, then rebase your local commits on top of the remote ones instead of creating a merge commit.
<code>git config --global pull.rebase true</code>	Make every <code>git pull</code> rebase by default.
<code>git config --global rebase.autoStash true</code>	Always autostash when rebasing.

<code>git push --force-with-lease</code>	Needed after rebasing a branch you had already pushed, because its commits were replaced. Refuses to overwrite work someone else pushed meanwhile.
<code>git reset --hard ORIG_HEAD</code>	Undo a rebase you just finished. <code>ORIG_HEAD</code> is where the branch pointed before.
<code>git reflog</code>	Find the pre-rebase position later on, then <code>git reset --hard HEAD@{n}</code> .

Conflicts during a rebase

Because commits are replayed one at a time, Git may stop more than once: each commit that clashes with the new base pauses the rebase. Every time, do the same three things. You do **not** run `git commit`.

```
git status                # see which files are in conflict
# edit the files, remove the <<<<<< ===== >>>>>> markers
git add <file>
git rebase --continue      # Git moves on to the next commit
```

Note: During a rebase the meaning of "ours" and "theirs" is the reverse of a merge: **ours** is the branch you are rebasing onto (for example `main`), and **theirs** is your own commit being replayed.

Interactive rebase: clean up commits before sharing them

`git rebase -i` opens an editor listing your commits, oldest first. Change the keyword in front of each line, save and close, and Git rewrites the history accordingly.

```
pick    a1b2c3d Add login form
squash  d4e5f6a Fix typo in form      # melt into the commit above
reword  789abcd Add validation        # keep changes, edit the message
drop    0f1e2d3 Debug print          # remove this commit
```

Keyword	Effect
<code>pick</code>	Keep the commit as it is
<code>reword</code>	Keep the changes, change the message
<code>edit</code>	Stop at this commit so you can change its content, then <code>git rebase --continue</code>
<code>squash</code>	Merge into the previous commit and combine both messages
<code>fixup</code>	Merge into the previous commit and discard this message
<code>drop</code>	Delete the commit
(reorder lines)	Commits are applied in the order they appear, so moving a line reorders history

<code>git rebase -i HEAD~3</code>	Edit the last 3 commits.
<code>git rebase -i main</code>	Edit every commit on your branch that is not yet on <code>main</code> .
<code>git rebase -i --root</code>	Edit the whole history, including the very first commit.
<code>git commit --fixup <commit></code>	Make a small correction commit that is marked to be folded into an earlier commit.
<code>git rebase -i --autosquash main</code>	Automatically place those fixup commits under their targets and squash them.

Moving a branch to a different base with --onto

Suppose `feature-b` was started from `feature-a`, but it should depend only on `main`. `--onto` takes the commits that are on `feature-b` but not on `feature-a` and replays them onto `main`.

```
git rebase --onto main feature-a feature-b
```

Form: `git rebase --onto <new-base> <old-base> <branch>`.

Merge or rebase?

	Merge	Rebase
History	Preserved exactly, with a merge commit	Rewritten into a straight line
Commit hashes	Unchanged	Changed for every replayed commit
Conflicts	Resolved once, in the merge commit	Possibly once per replayed commit
Shows when branches joined	Yes	No
Safe on shared branches	Yes	No
Best for	Bringing a finished feature into <code>main</code>	Updating or tidying your own unpublished branch

- Rebase when you want the latest `main` under your own feature branch before a pull request, or when you want to squash messy "wip" commits into one.
- Merge when the branch is shared with other people, or when you are not sure. Merging never rewrites anything.

Careful: Golden rule: never rebase commits that other people have already pulled. Rebase replaces those commits with new ones, so teammates who still hold the old ones get duplicates and conflicts on their next pull.

STEP 17

Rebase before a pull request

Use this when `main` on GitHub has moved on while you worked on your own branch (`new_branch` below). It puts your commits on top of the latest `main`, so the pull request shows only your changes and merges cleanly.

```
git branch                                # 1. confirm you are on new_branch (marked with *)
git remote -v                             # 2. confirm origin is the right GitHub repository
git add .                                  # 3. commit your work first:
git commit -m "Describe the change"        #   a rebase needs a clean working tree
git fetch origin                           # 4. download the latest commits (files untouched)
git rebase origin/main                     # 5. replay your commits on top of the latest main
git push origin new_branch                 # 6. upload the branch (first push)
```

```
git push --force-with-lease origin
new_branch
```

Use this for step 6 if the branch was **already on GitHub** before the rebase. Its commits were re-created, so a plain push is rejected as non-fast-forward.

```
git rebase --continue
```

Only when step 5 stopped on a conflict: fix the file, `git add` it, then continue. If Git already said "Successfully rebased and updated", this prints "No rebase in progress?", which is harmless.

```
git rebase --autostash origin/main
```

Alternative to step 3: rebase without committing first. Git stashes your uncommitted changes and restores them afterwards.

Then open the repository on GitHub. A new branch shows a **Compare & pull request** button: choose base `main`, compare `new_branch`, and create the PR. If the PR already exists, it updates by itself as soon as you push.

Tip: This is block 3 and block 4 of the daily development workflow at the front of these notes. Do it every time before you push, and again whenever a reviewer tells you the branch is out of date or has conflicts with `main`.

PART 5

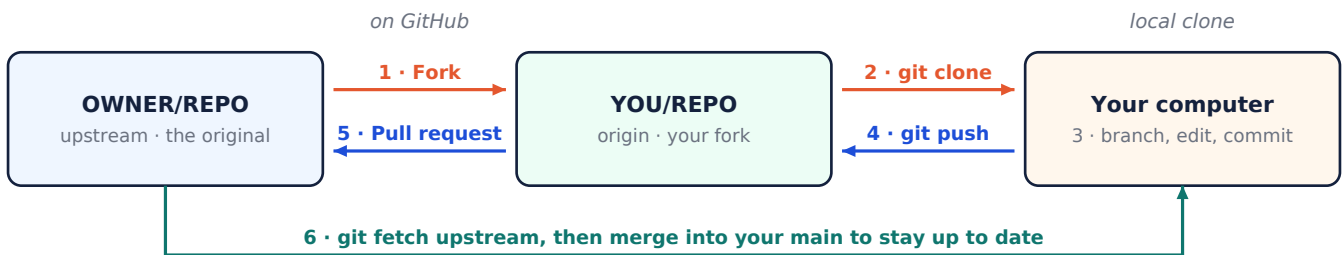
Pull requests and GitHub

Steps 18 to 22: collaboration, reviews, GitHub CLI, Actions, other features

STEP 18

Collaborate: fork, branch, pull request

There are two collaboration models. In a **shared repository** every teammate has write access, pushes branches to the same repository and opens pull requests there. In the **fork and pull** model (used by open source) you have no write access, so you work in your own fork and propose changes from it.



Fork workflow, start to finish

```
# 1. On GitHub press "Fork" (or: gh repo fork OWNER/REPO --clone)
git clone https://github.com/YOU/REPO.git # 2. clone YOUR fork
cd REPO
git remote add upstream https://github.com/OWNER/REPO.git
git remote -v # origin = your fork, upstream = original
git switch -c fix/typo-in-readme # 3. branch for the change
# ... edit, then ...
git add . && git commit -m "Fix typo in README"
git push -u origin fix/typo-in-readme # 4. push to your fork
# 5. On GitHub: "Compare & pull request"
# base = OWNER:main compare = YOU:fix/typo-in-readme
```

Keep your fork up to date

```
git fetch upstream # 6. get the latest commits from the original
git switch main
git merge upstream/main # or: git rebase upstream/main
git push origin main # update your fork on GitHub
```

git push	After review comments: commit the fixes on the same branch and push. The pull request updates itself.
git fetch origin pull/42/head:pr-42	Download pull request number 42 into a local branch pr-42 to test it.
git push origin --delete fix/typo-in-readme	Remove the branch on GitHub after the PR is merged.
git branch -d fix/typo-in-readme	Remove the local branch too.

Merge button on a PR	What it does	Use when
Create a merge commit	Keeps every commit and adds a merge commit	You want the full branch history

Merge button on a PR	What it does	Use when
Squash and merge	Turns the whole PR into one commit on <code>main</code>	The branch has many small "wip" commits
Rebase and merge	Replays each commit on <code>main</code> , no merge commit	You want linear history with individual commits

Tip: Open a **draft** pull request early to show work in progress. Keep PRs small and focused on one thing; they are reviewed faster and merged sooner.

Pull request checklist

- The title says what the PR does; the description says why, and how to test it.
- The issue is linked with `Closes #12`.
- The branch is rebased on the latest `main` and GitHub shows "no conflicts".
- Tests pass on your machine and the checks on GitHub are green.
- The diff contains only this task: no debug prints, no stray files, no secrets.
- A reviewer is requested, and every review comment gets an answer or a fix.
- After the merge the branch is deleted, on GitHub and locally.

STEP 19

Issues, reviews and repository settings

Issues track bugs, ideas and tasks. Each has a number (`#12`), and can carry labels, assignees and a milestone. Write one of these keywords followed by the issue number in a PR description or commit message, and GitHub closes the issue automatically when the PR is merged into the default branch:

`Closes #12` `Fixes #12` `Resolves #12`

Code review

- In the PR open **Files changed**, hover a line and press `+` to comment on it. You can also propose an exact replacement with a suggestion block, which the author applies with one click.
- Finish with **Review changes**: Comment, Approve, or Request changes.
- The author pushes new commits to answer the review, then resolves the conversations.

Protect the main branch (Settings, Branches or Rules)

- Require a pull request before merging, with a minimum number of approvals.
- Require status checks (tests from GitHub Actions) to pass.
- Block force pushes and branch deletion. Optionally require linear history or signed commits.

Setting	Options
Visibility	Public (anyone can see) or Private (only people you invite). Organisations on Enterprise also have Internal.
Access on personal repositories	Owner plus invited collaborators (Settings, Collaborators).
Roles in organisations	Read, Triage, Write, Maintain, Admin.

Markdown used in README files, issues and pull requests

You type	You get
# Title ## Section	Headings
bold <i>italic</i> ~~strike~~	Bold, italic, strikethrough
`code`	Inline code
```python ... ```	Code block with syntax highlighting
- item 1. item	Bullet list, numbered list
- [ ] task - [x] done	Checklist
[text](https://url)	Link
![alt](image.png)	Image
> quote	Block quote
a   b   then  --- ---	Table
@username #12	Mention a person, link an issue or PR

### STEP 20

## GitHub CLI (gh)

**gh** brings GitHub itself to the terminal: repositories, pull requests, issues, releases and workflow runs.

winget install --id GitHub.cli	Install on Windows. macOS: <b>brew install gh</b> . Ubuntu/Debian: <b>sudo apt install gh</b> .
gh auth login	Sign in to GitHub.
gh repo create	Create a repository interactively.
gh repo clone OWNER/REPO	Clone without typing the full URL.
gh repo fork OWNER/REPO --clone	Fork a repository and clone your fork; <b>upstream</b> is set for you.
gh repo sync YOU/REPO	Update your fork on GitHub from the original.
gh repo view --web	Open the current repository in the browser. <b>gh browse</b> does the same.
gh issue create --title "Bug" --body "Steps"	Open an issue.
gh issue list	List open issues. <b>gh issue view 12</b> shows one.
gh issue close 12	Close an issue.
gh pr create --fill	Open a pull request from the current branch, using your commit messages as title and body.
gh pr list	List open pull requests.
gh pr status	Pull requests that involve you.
gh pr checkout 42	Check out pull request 42 locally to test it.
gh pr diff 42	Show the changes of a pull request.
gh pr review 42 --approve	Approve. Also <b>--comment</b> and <b>--request-changes</b> .
gh pr merge 42 --squash --delete-branch	Squash-merge the PR and delete its branch.
gh run list	Recent GitHub Actions runs. <b>gh run watch</b> follows one live.

<code>gh workflow run ci.yml</code>	Start a workflow manually (it needs a <code>workflow_dispatch</code> trigger).
<code>gh release create v1.0.0 --generate-notes</code>	Publish a release with automatic release notes.
<code>gh gist create notes.py</code>	Share a single file as a gist.

## STEP 21

# GitHub Actions (CI/CD)

Actions run scripts on GitHub's servers whenever something happens in the repository, for example running your tests on every push. A workflow is a YAML file inside `.github/workflows/`.

Concept	Meaning
Workflow	One automated process, defined in one <code>.yml</code> file.
Event ( <code>on:</code> )	What triggers it: <code>push</code> , <code>pull_request</code> , <code>schedule</code> , <code>workflow_dispatch</code> (manual button).
Job	A group of steps that run on the same machine. Jobs run in parallel by default.
Step	One command ( <code>run:</code> ) or one reusable action ( <code>uses:</code> ).
Runner	The machine that executes the job: <code>ubuntu-latest</code> , <code>windows-latest</code> , <code>macos-latest</code> .
Secret	An encrypted value such as an API key, read as <code>\${{ secrets.NAME }}</code> .

## Example: `.github/workflows/ci.yml`

```
name: CI # name shown in the Actions tab

on: # when to run
 push:
 branches: [main]
 pull_request:
 branches: [main]

jobs:
 test: # job id
 runs-on: ubuntu-latest # runner
 steps:
 - uses: actions/checkout@v4 # download the repository code
 - uses: actions/setup-python@v5 # install Python
 with:
 python-version: "3.12"
 - run: pip install -r requirements.txt
 - run: pytest # a failing test marks the commit with a red X
```

- Commit and push this file. Results appear in the **Actions** tab and as a tick or cross next to each commit and PR.
- Add secrets under **Settings, Secrets and variables, Actions**. Never write keys directly in the YAML.
- Action versions move on; check each action's page for its current major version before copying an old tag.

## Other GitHub features

Feature	What it gives you
Pages	Free hosting for a static website from your repository. Settings, Pages, choose branch <code>main</code> and folder <code>/</code> ( <code>root</code> ); the site appears at <a href="https://USERNAME.github.io/REPO/">https://USERNAME.github.io/REPO/</a> .
Releases	Downloadable versions built on tags, with release notes and attached files.
Projects	Kanban boards and tables to plan issues and pull requests.
Discussions	A forum for questions and ideas that are not bugs.
Wiki	Longer documentation pages attached to the repository.
Gists	Share a single snippet or file, with its own history.
Codespaces	A full VS Code development environment in the browser.
github.dev	Press the <code>.</code> key on any repository to open a lightweight web editor.
Dependabot	Security alerts for vulnerable dependencies and automatic update pull requests.
Secret scanning	Detects committed tokens and keys; push protection can block the push.
Code scanning	Finds security bugs in the code with CodeQL.
Insights	Contributors, commit activity, traffic and the dependency graph.
Template repositories	Start new projects from a prepared skeleton with "Use this template".
Profile README	A repository named exactly like your username shows its README on your profile.
Student Developer Pack	Free tools and GitHub Pro features for verified students ( <a href="https://education.github.com">education.github.com</a> ).
Star / Watch / Fork	Bookmark a repository, get notifications from it, or copy it to your account.

## PART 6

# Advanced tools and reference

Steps 23 to 26: power commands, team conventions, errors and fixes, cheat sheet

## STEP 23

## Advanced Git tools

### Cherry-pick: copy one commit to another branch

<code>git cherry-pick &lt;commit&gt;</code>	Apply the changes of one commit on top of the current branch.
<code>git cherry-pick A^..B</code>	Apply a range of commits, from A to B inclusive.
<code>git cherry-pick -n &lt;commit&gt;</code>	Apply the changes but do not commit yet.
<code>git cherry-pick --abort</code>	Cancel. <code>--continue</code> after fixing a conflict.

### Bisect: find the commit that introduced a bug

```
git bisect start
git bisect bad # the current commit is broken
git bisect good v1.0.0 # this older version worked
Git checks out a commit in the middle. Test it, then answer:
git bisect good # or: git bisect bad (repeat until Git names the culprit)
git bisect reset # return to where you started
```

<code>git bisect run pytest</code>	Let Git do the whole search automatically, using a command that fails on bad commits.
------------------------------------	---------------------------------------------------------------------------------------

### Worktrees: two branches checked out at once

<code>git worktree add ../hotfix hotfix/login</code>	Check out another branch in a second folder, without stashing your current work.
<code>git worktree list</code>	List all worktrees.
<code>git worktree remove ../hotfix</code>	Remove a worktree when you are done.

### Submodules: a repository inside a repository

<code>git submodule add &lt;url&gt; libs/tool</code>	Add another repository as a subfolder, pinned to a specific commit.
<code>git clone --recurse-submodules &lt;url&gt;</code>	Clone a project together with its submodules.
<code>git submodule update --init --recursive</code>	Fetch submodules after a normal clone.
<code>git submodule update --remote</code>	Move submodules to the latest commit of their branch.

### Large files

GitHub warns about files above 50 MB and rejects files above 100 MB. Git LFS stores big binaries (models, datasets, videos) outside the normal history and keeps a small pointer in the repository.

<code>git lfs install</code>	Enable Git LFS once per machine (install the <code>git-lfs</code> package first).
------------------------------	-----------------------------------------------------------------------------------

<code>git lfs track "*.h5"</code>	Store all <b>.h5</b> files with LFS. This writes a rule into <b>.gitattributes</b> .
<code>git add .gitattributes</code>	Commit the rule so teammates get it too.
<code>git lfs ls-files</code>	List the files managed by LFS.
<code>git clone --filter=blob:none &lt;url&gt;</code>	Partial clone: download file contents only when needed. Helps with huge repositories.

## Aliases: shorter commands

<code>git config --global alias.st status</code>	Now <b>git st</b> runs <b>git status</b> .
<code>git config --global alias.co checkout</code>	<b>git co</b>
<code>git config --global alias.br branch</code>	<b>git br</b>
<code>git config --global alias.cm "commit -m"</code>	<b>git cm "message"</b>
<code>git config --global alias.unstage "restore --staged"</code>	<b>git unstage &lt;file&gt;</b>
<code>git config --global alias.last "log -1 HEAD"</code>	<b>git last</b> shows the most recent commit.
<code>git config --global alias.lg "log --oneline --graph --decorate --all"</code>	<b>git lg</b> draws the full branch graph.

## Hooks, signing, search and maintenance

<code>.git/hooks/pre-commit</code>	An executable script here runs before every commit (lint, tests). A non-zero exit cancels the commit. Others: <b>commit-msg</b> , <b>pre-push</b> .
<code>git commit --no-verify</code>	Skip the hooks once.
<code>git config --global gpg.format ssh</code>	Sign commits with your SSH key instead of GPG.
<code>git config --global user.signingkey ~/.ssh/id_ed25519.pub</code>	Choose the key used for signing. Add the same key on GitHub as a <b>signing key</b> .
<code>git config --global commit.gpgsign true</code>	Sign every commit automatically; GitHub shows them as "Verified". Single commit: <b>git commit -S</b> .
<code>git grep "TODO"</code>	Search the tracked files for text. Add <b>-n</b> for line numbers.
<code>git archive --format=zip -o project.zip HEAD</code>	Export the current version as a zip file without the <b>.git</b> folder.
<code>git gc</code>	Clean up and compress the repository database.
<code>git fsck</code>	Check the repository for corruption.
<code>git count-objects -vH</code>	Show how much disk space the repository uses.
<code>git filter-repo --path secrets.env --invert-paths</code>	Erase a file from the <b>entire</b> history. Needs the separate <b>git-filter-repo</b> tool; everyone must re-clone afterwards.

## STEP 24

# Team workflows and conventions

Strategy	How it works	Fits
GitHub Flow	<code>main</code> is always deployable. Every change is a short-lived branch plus a pull request.	Most teams and student projects
Git Flow	Long-lived <code>main</code> and <code>develop</code> , plus <code>feature/*</code> , <code>release/*</code> and <code>hotfix/*</code> branches.	Products with scheduled, versioned releases
Trunk-based	Everyone merges tiny changes into <code>main</code> at least daily; unfinished features hide behind flags.	Teams with strong automated testing

## Conventional commit messages

<type>(<optional scope>): <short summary in the imperative>

```
feat(auth): add password reset by email
fix(parser): handle empty date field
docs: update installation steps in README
refactor(api): split user service into modules
test: add unit tests for tokenizer
chore: bump pandas to 2.2
```

Type	Use for	Type	Use for
<code>feat</code>	A new feature	<code>fix</code>	A bug fix
<code>docs</code>	Documentation only	<code>style</code>	Formatting, no logic change
<code>refactor</code>	Restructuring without changing behaviour	<code>perf</code>	Performance improvement
<code>test</code>	Adding or fixing tests	<code>chore</code>	Maintenance, dependencies
<code>ci</code>	Pipeline and workflow files	<code>build</code>	Build system, packaging

## Habits that prevent most problems

- Commit small and often. One commit should do one thing.
- Run `git status` and `git diff --staged` before every commit, so you know exactly what goes in.
- Pull before you start work and before you push.
- One branch per task. Do not work directly on `main` in a team.
- Never commit passwords, API keys or `.env` files. Add them to `.gitignore` on day one.
- Never force-push or rebase a branch that other people use.
- Write a README. A repository without one looks unfinished.
- Tag releases, and protect `main` with required reviews and checks.

## STEP 25

# Common errors and how to fix them

Message you see	Why	Fix
<code>fatal: not a git repository</code>	You are not inside a project folder that has <code>.git</code> .	<code>cd</code> into the project, or run <code>git init</code> .
<code>Please tell me who you are</code>	Name and email are not configured.	Set <code>user.name</code> and <code>user.email</code> (Step 3).

Message you see	Why	Fix
nothing added to commit but untracked files present	Nothing is staged.	<code>git add &lt;file&gt;</code> first, then commit.
error: src refspec main does not match any	There are no commits yet, or the branch is called <code>master</code> .	Make a first commit, then <code>git branch -M main</code> .
fatal: remote origin already exists	<code>origin</code> is already defined.	<code>git remote set-url origin &lt;url&gt;</code>
fatal: The current branch has no upstream branch	The branch has never been pushed.	<code>git push -u origin &lt;branch&gt;</code>
! [rejected] main -> main (fetch first / non-fast-forward)	GitHub has commits that you do not have locally.	<code>git pull --rebase origin main</code> , resolve any conflict, then <code>git push</code> .
fatal: refusing to merge unrelated histories	Local and remote repositories were started separately (for example a README created on GitHub).	<code>git pull origin main --allow-unrelated-histories</code>
Support for password authentication was removed / Authentication failed	You typed your account password, or the token expired.	Use a personal access token, SSH, or <code>gh auth login</code> (Step 4).
Permission denied (publickey)	GitHub does not know your SSH key, or it is not loaded.	<code>ssh-add ~/.ssh/id_ed25519</code> , add the <code>.pub</code> key on GitHub, test with <code>ssh -T git@github.com</code> .
remote: Permission to X denied to USER / 403	You have no write access to that repository.	Fork it and open a PR, or ask to be added as a collaborator.
Your local changes would be overwritten by merge / checkout	You have uncommitted changes in files that are about to change.	Commit them, or <code>git stash</code> , do the pull or switch, then <code>git stash pop</code> .
CONFLICT (content): Merge conflict in file	Both sides changed the same lines.	Edit the file, <code>git add</code> , <code>git commit</code> . Or give up with <code>git merge --abort</code> (Step 13).
You are in 'detached HEAD' state	You checked out a commit or tag instead of a branch.	Keep the work: <code>git switch -c &lt;name&gt;</code> . Leave: <code>git switch main</code> .
warning: LF will be replaced by CRLF	Line-ending conversion on Windows.	Harmless. Controlled by <code>core.autocrlf</code> (Step 3).
File is 120 MB; this exceeds GitHub's file size limit of 100 MB	A large file is in one of your commits.	<code>git rm --cached &lt;file&gt;</code> , <code>git commit --amend</code> , then track it with Git LFS (Step 23).
Could not resolve host: github.com	No internet, a proxy, or a typo in the URL.	Check the connection and <code>git remote -v</code> .
(no message) Committed on the wrong branch	You forgot to switch branches first.	<code>git branch &lt;new&gt;</code> , <code>git reset --hard HEAD~1</code> , <code>git switch &lt;new&gt;</code> . The commit now lives only on the new branch. Commit or stash other changes first.
(no message) Deleted a branch or lost commits	A reset, rebase or delete went too far.	<code>git reflog</code> to find the hash, then <code>git branch &lt;name&gt; &lt;hash&gt;</code> .
(no message) Pushed a password or API key	A secret was committed.	Revoke or rotate the key <b>first</b> . Then remove it from history with <code>git filter-repo</code> and force-push.

**Tip:** When stuck, run `git status`. It nearly always tells you what state you are in and which command comes next.

# One-page cheat sheet

## Daily workflow

<code>git switch main   git pull</code>	Start from latest main
<code>git switch -c feature/x</code>	Branch for the task
<code>git add .   git commit -m "msg"</code>	Save your work
<code>git fetch origin</code>	Get teammates' work
<code>git rebase origin/main</code>	Put yours on top
<code>git push -u origin feature/x</code>	Upload, then open PR

## Setup

<code>git config --global user.name "N"</code>	Set your name
<code>git config --global user.email "E"</code>	Set your email
<code>git init</code>	New repository here
<code>git clone &lt;url&gt;</code>	Copy a repository
<code>git help &lt;command&gt;</code>	Open the manual

## Daily work

<code>git status</code>	What changed?
<code>git add &lt;file&gt;   git add .</code>	Stage changes
<code>git commit -m "msg"</code>	Save a snapshot
<code>git commit -am "msg"</code>	Stage tracked + commit
<code>git diff   git diff --staged</code>	Unstaged / staged changes
<code>git log --oneline --graph --all</code>	History as a graph

## Branches

<code>git branch</code>	List branches
<code>git switch -c &lt;name&gt;</code>	Create and switch
<code>git switch &lt;name&gt;</code>	Switch
<code>git merge &lt;name&gt;</code>	Merge into current
<code>git rebase main</code>	Replay on top of main
<code>git branch -d &lt;name&gt;</code>	Delete merged branch

## Stash and tags

<code>git stash   git stash -u</code>	Park changes
<code>git stash pop</code>	Bring them back
<code>git stash list</code>	Show stashes
<code>git tag -a v1.0.0 -m "msg"</code>	Create a release tag
<code>git push origin --tags</code>	Upload tags

## Inspect

<code>git show &lt;commit&gt;</code>	One commit in detail
<code>git log -p -- &lt;file&gt;</code>	History of a file
<code>git blame &lt;file&gt;</code>	Who changed each line
<code>git diff main..feature</code>	Compare branches
<code>git grep "text"</code>	Search the code

## Remote (GitHub)

<code>git remote -v</code>	List remotes
<code>git remote add origin &lt;url&gt;</code>	Connect to GitHub
<code>git push -u origin main</code>	First push
<code>git push</code>	Upload commits
<code>git pull</code>	Download and merge
<code>git fetch --all --prune</code>	Download only
<code>git push origin --delete &lt;b&gt;</code>	Delete remote branch

## Undo

<code>git restore &lt;file&gt;</code>	Discard changes
<code>git restore --staged &lt;file&gt;</code>	Unstage
<code>git commit --amend</code>	Fix the last commit
<code>git reset --soft HEAD~1</code>	Undo commit, keep work
<code>git reset --hard HEAD~1</code>	Undo commit, delete work
<code>git revert &lt;commit&gt;</code>	Safe undo of pushed commit
<code>git reflog</code>	Find lost commits

## Fork sync

<code>git remote add upstream &lt;url&gt;</code>	Link the original
<code>git fetch upstream</code>	Get its new commits
<code>git merge upstream/main</code>	Update your main
<code>git push origin main</code>	Update your fork

## GitHub CLI

<code>gh auth login</code>	Sign in
<code>gh repo create</code>	New repository
<code>gh pr create --fill</code>	Open a pull request
<code>gh pr checkout 42</code>	Test a pull request
<code>gh pr merge --squash</code>	Merge it

## Escape hatches

<code>git merge --abort</code>	Cancel a merge
<code>git rebase --abort</code>	Cancel a rebase
<code>git cherry-pick &lt;commit&gt;</code>	Copy one commit here
<code>git clean -fd</code>	Delete untracked files